

ui.markdown 全面详解

`ui.markdown` 是 NiceGUI 中用于渲染 Markdown 内容的专用组件，基于 `markdown2` 解析器实现，支持标准 Markdown 语法及扩展功能（如代码块高亮、表格、流程图、LaTeX 公式等）。该组件继承自 `ui.element` 基类，具备样式自定义、内容动态更新、数据绑定等核心能力，适用于文档展示、代码示例、数据可视化等场景，是快速构建富文本界面的关键工具。

一、核心初始化参数

`ui.markdown` 的初始化参数简洁直观，主要用于配置 Markdown 内容及扩展功能：

参数名	类型与说明	默认值	关键注意事项
content	待渲染的 Markdown 文本（支持单行字符串或多行字符串）	-	核心必填参数，支持标准 Markdown 语法及扩展语法（需通过 <code>extras</code> 启用）
extras	<code>markdown2</code> 扩展插件列表（字符串数组）	<code>['fenced-code-blocks', 'tables']</code>	用于启用额外功能（如代码块、表格、流程图等），需与 <code>markdown2</code> 支持的扩展匹配

基础使用示例

```
from nicegui import ui

# 1. 简单 Markdown 渲染（默认支持代码块和表格）
ui.markdown('''
# Hello Markdown!
This is a bold text, this is italic, and this is a [link](https://nicegui.io).

- Unordered list item 1
- Unordered list item 2
  - Nested list item
''')
```

```
# 2. 启用额外扩展（如 LaTeX 公式）
ui.markdown(r'''
## LaTeX 公式示例
Euler's identity:  $e^{i\pi} = -1$ 

Quadratic formula:  $x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$ 
''', extras=['latex'])

ui.run()
```

二、核心功能与使用场景

1. 自动处理缩进，保持格式整洁

`ui.markdown` 会自动剥离每行内容的公共缩进，允许在代码中缩进 Markdown 文本以保持代码结构清晰，且不影响最终渲染效果。

示例：

```
from nicegui import ui

# 代码中缩进 Markdown 内容，渲染时自动对齐
ui.markdown('''
    ## 带缩进的 Markdown
    这行文本在代码中缩进了 4 个空格，但渲染时无缩进。

    # 这行文本缩进了 8 个空格（超出公共缩进 4 个）
    因此会被识别为代码块，保留缩进并语法高亮。

    - 列表项 1（代码中缩进 4 个空格）
    - 列表项 2
''')

ui.run()
```

渲染结果：

- 标题「带缩进的 Markdown」无缩进；
- 中间 8 空格缩进的内容被渲染为代码块；
- 列表项正常显示，无额外缩进。

2. 代码块语法高亮

默认启用 `fenced-code-blocks` 扩展，支持用三重反引号 (`````) 包裹代码块，指定语言后可实现语法高亮（依赖 `Pygments` 库）。

支持的语言：Python、JavaScript、Java、HTML、CSS 等（完整列表见 [Pygments 官方文档](#)）。

示例：

```
from nicegui import ui

ui.markdown('''
## Python 代码示例
```python
from nicegui import ui

def greet(name: str) -> str:
 return f"Hello, {name}!"

ui.label(greet("NiceGUI")).classes("text-xl")
ui.run(dark=True)
''')
```

```
JavaScript 代码示例
function add(a, b) {
 return a + b;
}
console.log(add(2, 3)); // 输出 5

'''

ui.run()
```

### 3. 表格渲染

默认启用 `tables` 扩展，支持标准 Markdown 表格语法，可快速展示结构化数据。

示例：

```
```python
from nicegui import ui

ui.markdown('''
## 科学家列表
| 姓名          | 领域          | 国籍    |
|-----|-----|-----|
| 马克斯·普朗克 | 量子力学      | 德国    |
| 玛丽·居里     | 放射性研究    | 波兰/法国 |
| 阿尔伯特·爱因斯坦 | 相对论      | 德国/美国 |
''')

ui.run()
```

4. 流程图与图表（Mermaid 扩展）

通过启用 `mermaid` 扩展，支持 Mermaid 语法渲染流程图、时序图、饼图等，无需额外引入图表库。

示例：

```
from nicegui import ui

ui.markdown('''
## 流程图示例
```mermaid
graph TD
 A[开始] --> B[处理步骤 1]
 B --> C{判断条件}
 C -->|是| D[结果 A]
 C -->|否| E[结果 B]
 D --> F[结束]
 E --> F
'''

饼图示例
''', extras=['mermaid'])
```

```
ui.run()
```

## 5. LaTeX 公式渲染

启用 `latex` 扩展后，支持 inline 公式（`$公式$`）和 block 公式（`$$公式$$`），需安装依赖包：

```
pip install markdown2>=2.5 latex2mathml
```

示例：

```
from nicegui import ui

ui.markdown(r'''
数学公式示例
行内公式
勾股定理：$a^2 + b^2 = c^2$，其中 c 为斜边。

块级公式
傅里叶变换：
$$
\hat{f}(\xi) = \int_{-\infty}^{\infty} f(x) e^{-2\pi i \xi x} dx
$$

矩阵：
$$
A = \begin{pmatrix}
1 & 2 & 3 \\
4 & 5 & 6 \\
7 & 8 & 9
\end{pmatrix}
$$
''', extras=['latex'])

ui.run()
```

## 6. 动态更新内容

支持通过 `content` 属性或 `set_content` 方法动态修改 Markdown 内容，实现交互性富文本展示。

示例：

```
from nicegui import ui

创建 Markdown 组件并保存引用
md = ui.markdown('''
初始内容
点击下方按钮切换内容...
''')

按钮 1: 切换到「文档模式」
```

```
def show_doc():
 md.set_content('''
文档模式
这是动态加载的文档内容：
- 功能 1：支持 Markdown 所有标准语法
- 功能 2：动态更新无需刷新页面
- 功能 3：可结合其他组件交互
''')

按钮 2：切换到「代码模式」
def show_code():
 md.set_content('''
代码模式
```python
# 动态更新示例代码
print("Hello, Dynamic Markdown!")
''')

# 按钮布局
with ui.row():
    ui.button('显示文档', on_click=show_doc)
    ui.button('显示代码', on_click=show_code)
ui.run()
```

7. 样式自定义

`ui.markdown` 渲染的内容会自动添加 `nicegui-markdown` CSS 类，可通过自定义 CSS 调整字体、颜色、链接样式等；也可通过 `classes` 和 `style` 属性调整组件整体样式。

示例：

```
from nicegui import ui

# 1. 自定义 Markdown 内部元素样式（通过全局 CSS）
ui.add_css('''
/* 链接样式：橙色、无下划线，hover 时下划线 */
.nicegui-markdown a {
    color: #ff7e00;
    text-decoration: none;
}
.nicegui-markdown a:hover {
    text-decoration: underline;
}
/* 标题样式：深蓝色 */
.nicegui-markdown h2 {
    color: #1e40af;
    border-bottom: 2px solid #e2e8f0;
    padding-bottom: 0.3em;
}
/* 代码块样式：深色背景、圆角 */
.nicegui-markdown pre {
    background-color: #1f2937;
    border-radius: 0.5rem;
}
```

```
        padding: 1rem;
    }
'''

# 2. 调整组件整体样式（内边距、阴影）
ui.markdown('''
    ## 自定义样式示例
    - 链接: [NiceGUI 官网](https://nicegui.io)
    - 代码块:
    print("Custom style!")
''').classes('p-6 bg-gray-50 rounded-lg shadow-sm')

ui.run()
```

三、组件属性

`ui.markdown` 继承自 `ui.element`，除基础属性外，新增 `content` 绑定属性，支持动态同步内容：

属性名	类型	说明	适用场景
classes	Classes[Self]	组件的 CSS 类（Tailwind/Quasar 类），用于整体样式调整	组件布局、背景、边距配置
style	Style[Self]	内联 CSS 样式，用于精细化样式调整	字体大小、颜色、对齐方式
content	BindableProperty	Markdown 内容（可绑定到其他对象属性，支持双向同步）	动态内容更新、状态同步
html_id	str	HTML DOM 元素 ID（NiceGUI 2.16.0+ 新增）	原生 JS 交互、CSS 选择器定位
visible	BindableProperty	组件可见性（布尔值，支持绑定）	条件显示/隐藏富文本内容
props	Props[Self]	组件的 Quasar props 或 HTML 属性	底层属性配置（如 <code>disabled</code> ）

属性使用示例（内容绑定）

```
from nicegui import ui

# 定义数据类，用于绑定
class AppState:
    def __init__(self):
        self.md_content = "# 绑定初始内容"

state = AppState()

# Markdown 内容双向绑定到 state.md_content
md = ui.markdown('').bind_content(state, 'md_content')
```

```
# 输入框修改内容，同步到 Markdown
ui.input(
    label="修改 Markdown 内容",
    value=state.md_content,
    on_change=lambda e: setattr(state, 'md_content', e.value)
).classes('w-full mt-4')

# 按钮修改 state，同步到 Markdown
ui.button('重置内容', on_click=lambda: setattr(state, 'md_content', "# 重置后的内容")).classes('mt-2')

ui.run()
```

四、核心方法

除继承 `ui.element` 的所有方法（如 `move`、`delete`、`tooltip` 等）外，`ui.markdown` 新增专属方法用于内容操作，同时扩展了数据绑定方法：

方法名	参数与说明	功能描述
<code>set_content(content)</code>	<code>content</code> : 新的 Markdown 文本	手动设置组件内容（动态更新）
<code>bind_content</code>	<code>target_object</code> : 目标对象； <code>target_name</code> : 属性名； <code>forward/backward</code> : 转换函数	内容双向绑定到目标对象属性（组件 ↔ 目标对象）
<code>bind_content_from</code>	<code>target_object</code> : 目标对象； <code>target_name</code> : 属性名； <code>backward</code> : 转换函数	内容单向绑定（目标对象 → 组件）
<code>bind_content_to</code>	<code>target_object</code> : 目标对象； <code>target_name</code> : 属性名； <code>forward</code> : 转换函数	内容单向绑定（组件 → 目标对象）

方法使用示例（单向绑定）

```
from nicegui import ui

# 滑块值控制 Markdown 内容
slider = ui.slider(min=1, max=6, value=3, label='标题级别')

# Markdown 标题级别单向绑定到滑块值
ui.markdown('').bind_content_from(
    slider, 'value',
    backward=lambda level: f"{'#' * level} 动态标题（级别 {level}）\n\n滑块值: {level}"
)

ui.run()
```

五、版本兼容性与依赖说明

功能 / 扩展	最低版本要求	依赖说明
基础 Markdown 渲染	-	内置 <code>markdown2</code> 依赖，无需额外安装
html_id 属性	2.16.0	新增 HTML DOM 元素 ID 配置能力
LaTeX 公式	-	需安装 <code>markdown2>=2.5</code> 和 <code>latex2mathml</code> （ <code>pip install markdown2 latex2mathml</code> ）
Mermaid 流程图	-	内置支持，无需额外依赖（依赖 <code>markdown2</code> 的 <code>mermaid</code> 扩展）
代码块高亮	-	内置 <code>Pygments</code> 依赖，支持多数编程语言
bind_content 方法	3.0.0	新增内容绑定方法，支持 <code>strict</code> 参数校验目标属性（3.0.0+）

六、常见问题与注意事项

- 扩展启用冲突：**`extras` 参数需传入 `markdown2` 支持的扩展名称（如 `['mermaid', 'latex']`），无效扩展会被忽略，需参考 [markdown2 官方扩展列表](#)；
- LaTeX 公式渲染失败：**确保 `markdown2` 版本 ≥ 2.5 ，且已安装 `latex2mathml`，公式语法需严格遵循 LaTeX 规范（如换行需用 `\\`，特殊字符需转义）；
- 代码块无高亮：**检查语言名称是否正确（如 `python` 而非 `Python`），确保 `Pygments` 支持该语言；
- 样式自定义优先级：**内联 `style` > 自定义 CSS（`nicegui-markdown` 类）> 组件默认样式，冲突时按优先级覆盖；
- 大文本性能：**渲染超长 Markdown 内容（如万字文档）时，建议分页加载，避免界面卡顿。

七、高级应用场景示例

1. 文档阅读器（结合标签页）

```
from nicegui import ui

# 文档内容字典
docs = {
    '简介': '''
# NiceGUI Markdown 组件
`ui.markdown` 是富文本渲染核心组件，支持：
- 标准 Markdown 语法
- 代码块高亮
- 表格、流程图、LaTeX 公式
- 动态内容更新
'''
    '使用指南': '''
# 使用指南
## 1. 基础用法
'''
}
```



```

```python
from nicegui import ui
ui.markdown('Hello **Markdown**!')
ui.run()

2. 启用扩展

```python
ui.markdown(content, extras=['mermaid', 'latex'])
'''

' 扩展列表 ': '''

# 支持的扩展

| 扩展名称          | 功能描述          |
| ----- | ----- |
| fenced-code-blocks | 代码块（默认启用） |
| tables            | 表格（默认启用）   |
| mermaid           | 流程图、图表       |
| latex             | LaTeX 公式         |
| strikethrough     | 删除线（`~~文本~~`） |
| footnote          | 脚注（`[^1]`）     |
| '''              |                    |
| }                 |                    |

# 标签页切换文档

with ui.tabs () as tabs:
    ui.tab (' 简介 ')
    ui.tab (' 使用指南 ')
    ui.tab (' 扩展列表 ')

with ui.tab_panels (tabs, value=' 简介 '):
    for tab_name, content in docs.items ():
        with ui.tab_panel (tab_name):

ui.markdown(content).classes ('p-4')

ui.run()

```

2. 交互式数学公式编辑器

```

from nicegui import ui

# 初始公式
initial_latex = r"e^{i\pi} = -1"

# 公式输入框
latex_input = ui.textarea(
    label="LaTeX 公式",
    value=initial_latex,

```

```
rows=3,
placeholder=r"输入 LaTeX 公式（如：a^2 + b^2 = c^2）"
).classes('w-full')

# 实时预览 Markdown
preview = ui.markdown(
    f"${n{initial_latex}}\n$$",
    extras=['latex']
).classes('p-4 bg-gray-50 rounded-lg')

# 输入框变化时更新预览
latex_input.on('input', lambda e: preview.set_content(f"${n{e.value}}\n$$"))

ui.run()
```

总结

`ui.markdown` 是 NiceGUI 中功能强大的富文本渲染组件，核心优势在于：

1. **语法兼容性强**：支持标准 Markdown 及多种扩展（代码块、表格、流程图、LaTeX 等），满足多样化富文本需求；
2. **易用性高**：自动处理缩进、内置样式优化，无需手动调整布局；
3. **动态交互**：支持内容动态更新和数据绑定，可结合其他组件实现交互式富文本场景；
4. **样式灵活**：支持全局 CSS 自定义和组件级样式配置，适配不同界面风格。

无论是文档展示、代码示例、数据可视化还是交互式富文本编辑，`ui.markdown` 都能提供简洁高效的解决方案，是 NiceGUI 界面开发中不可或缺的核心组件之一。